Bike-Sharing Station Rebalancing Optimizer: Greedy and Dynamic-Programming Transfer Planning with Parallel Scenario Evaluation

Kollipara Teja
Scholar, M.Tech
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
2601050118@kluniversity.in

Dr. Raju Anitha
Professor
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
anitharaju@kluniversity.in

***Abstract*—Bike-sharing stations in busy areas run out of bikes while others fill up, and operators correct this by moving bikes with a small fleet of capacity-limited trucks. This paper presents a Bike-Sharing Station Rebalancing Optimizer, a Python prototype that (i) converts predicted demand into integer target inventories that respect station capacity and conserve the bike total, (ii) plans transfers with a Greedy heuristic and with an exact Dynamic Programming (DP) solution of the underlying transportation problem, (iii) turns transfers into validated truck routes, and (iv) scores both plans with one objective combining truck distance, residual imbalance and trucks used. A multiprocessing runner evaluates independent scenarios in parallel, and a Streamlit dashboard presents routes and comparisons. On a 12-station example both methods removed the full imbalance of 62 bikes, while DP lowered transfer cost by 13.8% and truck distance by 12.8%. Over 30 seeded scenarios DP never cost more than Greedy (117.3 versus 127.8 bike-km on average), but it finished within its work budget only for smaller systems, whereas Greedy handled 50 stations in under one millisecond. On a two-core machine the parallel runner reached a 1.74× speed-up for 96 runs and was slower than one process for 16 runs. All data are synthetic, so the results illustrate the methods and are not claims about any real operator.**

***Index Terms*—bike sharing, static rebalancing, greedy algorithm, dynamic programming, transportation problem, route planning, multiprocessing.**

I. INTRODUCTION

Bike-sharing systems let people rent a bicycle at one station and return it at another [1], [2]. Use is rarely balanced: commuter destinations and transit hubs lose bikes in the morning and gain them in the evening, so some stations become empty and others full, which blocks trips from starting or ending. Recent surveys confirm that rebalancing and related service-planning decisions remain central operational problems [3], and that demand varies strongly with time, weather and location [4].

Operators address this by *rebalancing*: trucks collect bikes from stations with more than they need and deliver them to stations with less [5]. Rebalancing at night, when demand is low, is called *static* rebalancing and is the setting of this paper [6], [7]. The planner must decide which stations give bikes, which receive them, how many bikes move between each pair, and how limited trucks are routed with little driving.

Full routing formulations are hard to solve and hard to inspect. This work instead isolates the transfer-selection subproblem, solves it with a fast heuristic and with an exact method, and builds the rest of the pipeline (targets, truck routes, validation, scoring, parallel evaluation) around it so that the two methods can be compared fairly on identical inputs. The contributions are:

- a demand-proportional target model with water-filling and largest-remainder rounding that conserves the total number of bikes;
- Greedy and exact memoised Dynamic Programming optimizers behind one interface, with a work budget that reports oversized problems as *skipped*;
- a route planner that executes every plan on a copy of the inventory so that all constraints are verified independently of the optimizer;
- a single objective score and a serial/parallel scenario runner, with measured quality, scalability and speed-up results.

The paper is organised as follows. Section II reviews related work and background, Section III describes the architecture, Section IV details the methods, Section V gives the experimental setup, Section VI reports results, and Sections VII and VIII discuss limitations and conclude.

II. RELATED WORK AND BACKGROUND

A. Rebalancing Research

Chemla *et al.* showed that static rebalancing is NP-hard and proposed exact and heuristic methods [6]. Raviv *et al.* modelled static repositioning with user-dissatisfaction and cost terms [7], and Dell'Amico *et al.* provided formulations and benchmark instances [8]. These works combine transfers with full vehicle routing and are considerably harder than the transfer subproblem studied here.

More recent work has moved toward dynamic and data-driven control. Brinkmann develops approximate-dynamic-programming policies that decide inventory and routing actions while the system is operating [9]. Gammelli *et al.* show that more accurate demand forecasts do not necessarily yield better inventory decisions, so forecasts and decision models should be evaluated together [10]. Liu and Pelechrinis estimate unobserved (excess) demand to improve net-demand prediction [11], and Liang *et al.* use a dual-policy reinforcement-learning method that decouples inventory and routing decisions for real-time rebalancing [12]. In contrast with these dynamic methods, the present prototype is static and deterministic, and takes predicted demand as an input.

B. Targets and the Transportation Problem

A target inventory T_i states how many bikes station i should hold. Allotting bikes in proportion to predicted demand,

capping stations at capacity C_i and redistributing the excess (water-filling), then rounding by the largest-remainder rule, yields integer targets with $0 \le T_i \le C_i$ and $\Sigma T_i = \Sigma B_i$, where B_i is the current bike count. Stations above target have surplus s_i, those below have shortage n_j, and the total imbalance is

$$I = \Sigma_i |B_i - T_i|. \quad (1)$$

Choosing the number of bikes x_{ij} to send from surplus station i to shortage station j so that all shortages are met at minimum cost

$$\min \Sigma_{i,j} d_{ij} x_{ij}, \text{ s.t. } \Sigma_j x_{ij} \le s_i, \Sigma_i x_{ij} = n_j \quad (2)$$

is the classical transportation problem [13], [14], whose assignment special case is solved by the Hungarian method [15]. Here it is solved by dynamic programming because the method is exact for the integer model, easy to explain, and instructive about state-space growth.

C. Greedy and Dynamic Programming

A greedy algorithm repeatedly makes the locally best choice without revisiting it; it is fast but carries no optimality guarantee here. Dynamic programming combines optimal solutions of overlapping subproblems and memoises each result [16], [17]. For the transportation problem the state is the vector of remaining amounts, and its size can reach $\Pi(s_i + 1)$, which grows exponentially with the number of stations. This is why the implementation picks the smaller of the two possible state spaces and enforces a work budget.

D. Routing, Distance and Parallelism

Transfer cost $\Sigma d_{ij} x_{ij}$ counts bike-kilometres, not truck-kilometres: a truck carrying ten bikes five kilometres drives five kilometres, and may serve several stations per tour, which is the vehicle routing problem [18]. The prototype therefore separates transfer selection from route construction. Distances use the haversine formula [19]. Independent scenarios can be run on separate processes with Python's *multiprocessing* and process-pool facilities [20], [21]; speed-up is serial time over parallel time, and efficiency is speed-up divided by the number of workers.

TABLE I
COMPARISON OF APPROACHES USED IN THIS WORK

Approach	Strength	Limitation	Role here
Rule-based	Simple	No cost measure	Motivation
Greedy	Very fast	May miss cheaper pairing	Baseline
Dynamic prog.	Exact for model	State space grows	Exact optimizer
Full routing [6], [8]	Models tours	Hard to solve	Future work
Multiprocessing	Faster for big batches	Overhead on small ones	Parallel runner

III. SYSTEM ARCHITECTURE

The application has six layers (Fig. 1). The user interface (Streamlit dashboard [22] and command line) collects configuration. Data management generates seeded synthetic systems or loads CSV files and builds the distance matrix with NumPy and pandas [24], [25]. The demand analyzer computes targets and imbalance. The optimization engine contains the Greedy and DP optimizers, both returning a list of transfers. Route planning builds and validates truck routes and hosts the parallel runner. The results layer computes metrics and draws Plotly charts [23]. Algorithms are independent of the interface, so they can be tested without starting the dashboard.

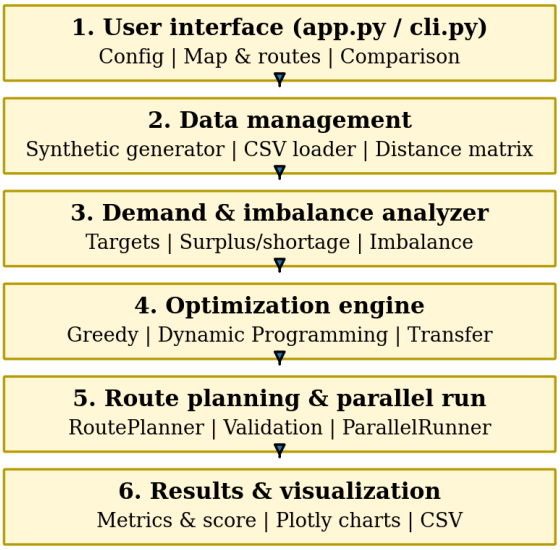


Fig. 1. Logical architecture of the rebalancing optimizer.

A scenario starts as a station table (coordinates, capacity, current bikes) and a predicted-demand table. A copy of the system is passed to the chosen optimizer, the route planner turns its transfers into trips, and the simulation engine executes the routes on the inventory, recomputes the final imbalance, and returns a result object.

IV. METHODOLOGY

A. Greedy Optimizer

The Greedy optimizer keeps a surplus map and a shortage map. While both are non-empty it selects the station with the largest remaining shortage, finds the nearest station that still has surplus, transfers q = min(surplus, need) bikes, and removes exhausted entries. Ties are broken by station index, so results are deterministic. Each iteration exhausts at least one entry, so the number of iterations is at most the number of surplus plus shortage stations, giving $O(k(m + k))$ time for m surplus and k shortage stations.

B. Dynamic Programming Optimizer

The DP optimizer treats one side as stages and stores the remaining amounts of the other side in the state. With $V(s)$ the minimum cost of serving all remaining shortages from state s,

$$V(s) = \min_q \{ d \cdot q + V(s') \}, \quad (3)$$

where q ranges over feasible quantities for the current pair and s' is the reduced state. A memo table stores the best value and best q of every state, and the plan is read back from the stored decisions. Two refinements keep the method usable: the optimizer chooses the smaller of Π(supply + 1) and Π(need + 1) as the state space (total supply equals total need, so the problem is symmetric), and a budget of three million transitions raises a *ProblemTooLargeError*, which the simulation engine reports as *skipped* rather than failing or hanging.

Worked example. Stations A and B each hold five surplus bikes; C and D each need five. Distances are A–C 4.33 km,

B–C 4.33 km, A–D 1.08 km and B–D 7.59 km. Greedy serves C first and, with A and B equally near, picks A, which forces B → D. DP finds that B → C and A → D is much cheaper (Table II): Greedy's first, apparently harmless, choice consumed the only cheap source for D.

TABLE II
WORKED EXAMPLE: GREEDY VERSUS DP

Plan	Transfers	Cost (bike-km)
Greedy	A→C (5), B→D (5)	5×4.33 + 5×7.59 = 59.60
DP	B→C (5), A→D (5)	5×4.33 + 5×1.08 = 27.09

C. Route Planning and Validation

Transfers larger than truck capacity are split into trips (drive to source, pick up, drive to destination, deliver). Trips are ordered by pickup distance from the depot and assigned to trucks in one of two modes: *minimum distance* (the truck whose route grows least, often a single truck) or *spread work* (the truck whose completed route would be shortest). Every route starts and ends at the depot. The routes are then *executed* on a copy of the inventory, and an error is raised if a pickup exceeds a station's bikes, a delivery exceeds capacity, a truck exceeds its load limit or finishes loaded, or more trucks than available are used. Because the check runs on the final plan, it protects against faults in any optimizer.

D. Objective Function

Both plans are scored identically:

$$\text{score} = \alpha D + \beta I_{\text{after}} + \gamma V, \tag{4}$$

where D is total truck distance (km), I_{after} the remaining imbalance and V the number of trucks dispatched. The defaults are $\alpha = 1$ per km, $\beta = 2$ per misplaced bike and $\gamma = 5$ per truck; lower is better. The weights express a policy: leaving one bike misplaced costs as much as driving two kilometres.

E. Parallel Scenario Evaluation

A scenario is a small specification (stations, trucks, capacity, seed, algorithm, weights). A module-level worker rebuilds the system from the seed, runs one algorithm and returns a flat dictionary of metrics, so no large objects cross process boundaries and serial and parallel runs perform identical work. The runner uses the *spawn* start method for portability and returns results in input order with the wall-clock time.

V. EXPERIMENTAL SETUP

All experiments use synthetic scenarios from fixed seeds, so they are exactly repeatable. Stations are placed uniformly in roughly an 8 km × 8 km area; capacity is an integer in [15, 30]; the initial fill fraction follows Beta(1.2, 1.2); and predicted demand is uniform in [0.3, 1.0], multiplied by 2–3.5 for about a quarter of stations to model hot spots. Experiments ran on a two-core Linux (x86-64) cloud sandbox with Python 3.13, NumPy 2.5 and pandas 3.0, using weights $\alpha = 1$, $\beta = 2$, $\gamma = 5$. The example scenario has 12 stations, 3 trucks, capacity 10 and seed 42. The quality experiment uses 10 stations and seeds 1–30; the scalability experiment uses 6–50 stations with seeds 1–2; the parallel experiment uses 12 stations with seeds from 1000, reporting the median of three repetitions. The DP work budget is 3,000,000 transitions. Correctness is supported by 23 automated tests, including agreement of DP with an independent exhaustive search on 60 random small cases.

VI. RESULTS AND DISCUSSION

A. Example Scenario

In the 12-station example the initial imbalance was 62 bikes, and both algorithms moved 31 bikes with three trucks and reduced the imbalance to zero. DP achieved a lower transfer cost (76.79 versus 89.05 bike-km, 13.8% lower), a shorter total truck distance (70.64 versus 81.04 km, 12.8% shorter) and a lower score (85.64 versus 96.04), at a longer optimization time (34.3 ms versus 0.30 ms) that remains negligible for planning (Table III, Fig. 2).

TABLE III
EXAMPLE SCENARIO (12 STATIONS, 3 TRUCKS)

Metric	Greedy	DP
Imbalance before / after	62 / 0	62 / 0
Bikes moved	31	31
Truck distance (km)	81.04	70.64
Transfer cost (bike-km)	89.05	76.79
Objective score	96.04	85.64
Optimization time	0.30 ms	34.3 ms

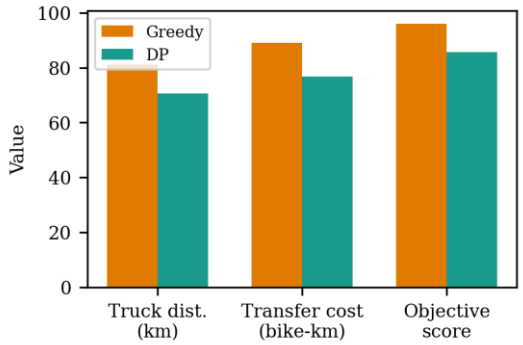


Fig. 2. Greedy versus DP on the 12-station example.

B. Quality over 30 Scenarios

To avoid concluding from one example, both algorithms ran on 30 seeded 10-station scenarios (mean initial imbalance 69.9 bikes). All 60 runs removed the full imbalance. DP's transfer cost was never higher than Greedy's (30 of 30) and strictly lower in 26; in the other 4 the costs were equal. On average DP reduced transfer cost by 8.2% (Table IV). Truck distance is not the quantity DP minimises, so its routes were shorter in only 17 of 30 scenarios, although on average the lower transfer cost translated into 5.4% shorter routes and a 4.5% lower score.

TABLE IV
MEAN OVER 30 RANDOM SCENARIOS

Measure	Greedy	DP
Transfer cost (bike-km)	127.8	117.3
Truck distance (km)	73.4	69.4
Objective score	88.4	84.4
Optimization time	0.18 ms	76.7 ms
Lower transfer cost	–	26 of 30
Shorter truck route	–	17 of 30

C. Scalability

Greedy's time stayed below one millisecond up to 50 stations, whereas DP grew by roughly an order of magnitude or more for every two additional stations, from 0.16 ms at 6 stations to 0.43 s at 12 and 2.6 s at 16 (Table V, Fig. 3). Both seeds

exceeded the work budget at 14, 18, 20, 30 and 50 stations. That 14 stations were skipped while 16 completed is not a contradiction: difficulty depends on how many surplus and shortage stations a scenario has and how large their amounts are, not on station count alone. The result agrees with the theory of Section II-C: the exact method is practical only for modest problems, and Greedy still returns a feasible plan instantly when an exact answer is impractical.

TABLE V
OPTIMIZATION TIME VERSUS NUMBER OF STATIONS

Stations	Greedy	DP	DP status (2 seeds)
6	0.13 ms	0.16 ms	both completed
8	0.14 ms	1 ms	both completed
10	0.15 ms	12 ms	both completed
12	0.25 ms	433 ms	both completed
14	0.22 ms	–	both skipped
16	0.25 ms	2.64 s	both completed
18–50	0.28–0.76 ms	–	both skipped

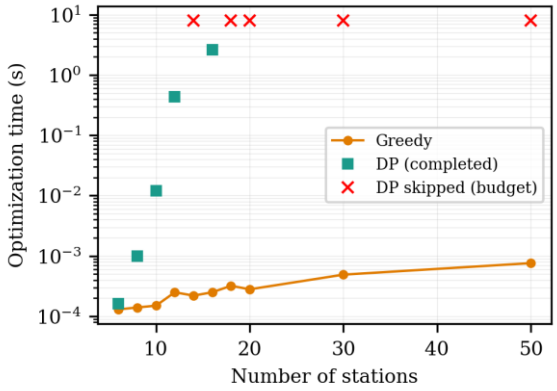


Fig. 3. Optimization time versus number of stations (log scale). Red crosses mark DP runs skipped at the work budget (plotted at an arbitrary height).

D. Truck-Balancing Option

Using Greedy transfers on 20 ten-station scenarios with three trucks, the minimum-distance strategy used on average 1.0 truck and drove 73.4 km, whereas spreading work used 3.0 trucks and drove 76.1 km (3.5% more) but cut the longest single route to 29.1 km (Table VI). The choice is a trade-off between total cost and completion time, so the dashboard exposes it as a switch.

TABLE VI
EFFECT OF THE TRUCK-BALANCING OPTION (MEAN OF 20)

Strategy	Trucks	Total km	Longest km
Minimum distance	1.0	73.4	73.4
Spread work	3.0	76.1	29.1

E. Parallel Evaluation

Batches of 16, 48 and 96 runs were executed with one and with two worker processes (median of three repetitions); serial and parallel results were identical in every batch, confirming reproducibility across processes (Table VII, Fig. 4). For 16 runs the parallel version was slower (0.46×) because process start-up and communication cost more than the 0.3 s of computation. For 48 and 96 runs the speed-ups were 1.50× and 1.74×, an efficiency of about 87% of the ideal factor of two. Parallelism therefore pays off only when the workload repays its overhead. These are measurements from one machine and should not be generalised.

TABLE VII
SERIAL VERSUS PARALLEL SCENARIO BATCHES

Runs	1 proc. (s)	2 proc. (s)	Speed-up	Effic.	Same
16	0.30	0.65	0.46×	23%	yes
48	3.09	2.07	1.50×	75%	yes
96	15.52	8.92	1.74×	87%	yes

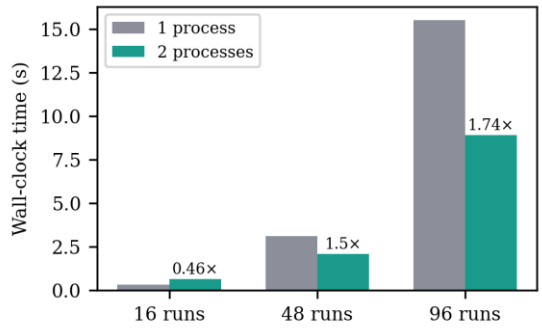


Fig. 4. Serial versus parallel wall-clock time (labels show speed-up).

F. Discussion

Four conclusions follow. First, both algorithms produce valid plans that remove the full imbalance, and the validation layer guarantees every constraint. Second, exact optimisation gives a lower transfer cost in most scenarios and never a higher one, with a measurable but modest benefit (about 8% on average). Third, that benefit costs computation time that grows exponentially with problem size, so Greedy remains the practical choice for large systems and interactive use, consistent with the trade-off between heuristic speed and exact quality noted in the rebalancing literature [6], [8]. Fourth, parallel execution helps for large batches only.

VII. LIMITATIONS AND FUTURE WORK

The model is static and single-night, with no time windows, driver shifts or demand during rebalancing. Each trip moves bikes from one station to one station; multi-stop loading is not modelled. Distances are great-circle, not road distances. DP optimises the simplified transfer cost rather than the full routing objective, and is practical only for small and medium problems. Predicted demand is an input, and all data are synthetic; timings come from one two-core machine.

Future work includes real station and trip data with a forecasting model in the spirit of [10], [11]; road distances and time windows; multi-stop tours following the routing formulations of [6], [8]; replacing exhaustive transition enumeration with a minimum-cost-flow or integer-programming solver; local-search improvement of Greedy plans; and extending the static planner toward the dynamic, learning-based policies of [9], [12], with confidence intervals over more seeds and machines.

VIII. CONCLUSION

This paper presented a tested Bike-Sharing Station Rebalancing Optimizer that computes demand-based targets, plans transfers with Greedy and exact DP methods, validates every plan, scores both with one objective, and evaluates scenarios in parallel. Both methods removed the full imbalance in every scenario; DP reduced transfer cost by

13.8% on the example and by 8.2% on average over 30 scenarios, but its running time exceeded the work budget for larger systems, while Greedy stayed below one millisecond at 50 stations. Parallel execution cut a 96-run batch time by a factor of 1.74 on two cores but was slower on a small batch. The prototype illustrates the central trade-off between fast heuristics and exact methods and the conditions under which parallelism helps.

REFERENCES

[1] P. DeMaio, "Bike-sharing: History, impacts, models of provision, and future," *J. Public Transp.*, vol. 12, no. 4, pp. 41–56, 2009.

[2] S. Shaheen, S. Guzman, and H. Zhang, "Bikesharing in Europe, the Americas, and Asia: Past, present, and future," *Transp. Res. Rec.*, no. 2143, pp. 159–167, 2010.

[3] C. S. Shui and W. Y. Szeto, "A review of bicycle-sharing service planning problems," *Transp. Res. C, Emerg. Technol.*, vol. 117, Art. no. 102648, 2020.

[4] E. Eren and V. E. Uz, "A review on bike-sharing: The factors affecting bike-sharing demand," *Sustain. Cities Soc.*, vol. 54, Art. no. 101882, 2020.

[5] G. Laporte, F. Meunier, and R. Wolfler Calvo, "Shared mobility systems," *4OR*, vol. 13, no. 4, pp. 341–360, 2015.

[6] D. Chemla, F. Meunier, and R. Wolfler Calvo, "Bike sharing systems: Solving the static rebalancing problem," *Discrete Optim.*, vol. 10, no. 2, pp. 120–146, 2013.

[7] T. Raviv, M. Tzur, and I. A. Forma, "Static repositioning in a bike-sharing system: Models and solution approaches," *EURO J. Transp. Logist.*, vol. 2, pp. 187–229, 2013.

[8] M. Dell'Amico, E. Hadjicostantinou, M. Iori, and S. Novellani, "The bike sharing rebalancing problem: Mathematical formulations and benchmark instances," *Omega*, vol. 45, pp. 7–19, 2014.

[9] J. Brinkmann, *Active Balancing of Bike Sharing Systems* (Lecture Notes in Mobility). Cham, Switzerland: Springer, 2020.

[10] D. Gammelli, Y. Wang, D. Prak, F. Rodrigues, S. Minner, and F. C. Pereira, "Predictive and prescriptive performance of bike-sharing demand forecasts for inventory management," *Transp. Res. C, Emerg. Technol.*, vol. 138, Art. no. 103571, 2022.

[11] X. Liu and K. Pelechrinis, "Excess demand prediction for bike sharing systems," *PLoS ONE*, vol. 16, no. 6, Art. no. e0252894, 2021.

[12] J. Liang, S. D. Jena, D. Liu, and A. Lodi, "Dual policy reinforcement learning for real-time rebalancing in bike-sharing systems," arXiv:2406.00868, 2024.

[13] F. L. Hitchcock, "The distribution of a product from several sources to numerous localities," *J. Math. Phys.*, vol. 20, pp. 224–230, 1941.

[14] G. B. Dantzig, *Linear Programming and Extensions*. Princeton, NJ, USA: Princeton Univ. Press, 1963.

[15] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logist. Quart.*, vol. 2, pp. 83–97, 1955.

[16] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 1957.

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[18] P. Toth and D. Vigo, Eds., *The Vehicle Routing Problem*. Philadelphia, PA, USA: SIAM, 2002.

[19] R. W. Sinnott, "Virtues of the Haversine," *Sky Telescope*, vol. 68, no. 2, p. 159, 1984.

[20] Python Software Foundation, "multiprocessing — Process-based parallelism," Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/multiprocessing.html

[21] Python Software Foundation, "concurrent.futures — Launching parallel tasks," Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/concurrent.futures.html

[22] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/

[23] Plotly Technologies Inc., "Plotly Python graphing library." [Online]. Available: https://plotly.com/python/

[24] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[25] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56–61.